Sim — **é totalmente possível** reutilizar suas *Pages atuais* migrando para o padrão **Screenplay usando ScreenPy**, inclusive como uma **evolução arquitetural mais sólida do que Behave puro**.

A diferença-chave:

- Behave → muda a forma de escrever testes (BDD)
- Screenplay (ScreenPy) → muda o **modelo de interação com o sistema (arquitetura de testes)**

Você pode inclusive **combinar Screenplay + Behave depois**, mas para a PoC faz sentido validar **Screenplay isoladamente primeiro**.

Abaixo segue um plano equivalente ao anterior, **adaptado para ScreenPy**, mantendo as mesmas restrições:

- NÃO modificar código existente
- Reutilizar Pages
- Evolução incremental
- Executável manualmente

---

1. PLANO DETALHADO – SCREENPLAY COM SCREENPY

1.1 Estratégia da PoC

Objetivo

- Introduzir padrão Screenplay usando ScreenPy
- Reutilizar:
  - pages/
  - fixtures/
  - data/
- Substituir:
  - transactions/ → por **Tasks (Screenplay)**
- Manter:
  - specs/ intactos

1.2 Conceito de Mapeamento (CRÍTICO)

Camada Atual	Screenplay
Page Object	Page (mantém)
Transaction	Task

Test (spec)	Scenario (Actor)
Fixture driver	Ability

1.3 Estrutura de Pastas NOVA

```
tests/
├── screenplay/
│   ├── actors/
│   │   └── actor_factory.py
│   │
│   ├── abilities/
│   │   └── browse_the_web.py
│   │
│   ├── tasks/
│   │   ├── login.py
│   │   ├── add_to_cart.py
│   │   └── checkout.py
│   │
│   ├── questions/
│   │   └── inventory.py
│   │
│   ├── interactions/
│   │   └── (opcional)
│   │
│   └── tests/
│       ├── test_login_screenplay.py
│       └── test_checkout_screenplay.py
│
tests/docs/
└── screenplay_poc.md
```

1.4 Dependências

Adicionar no requirements.txt:

```
screenpy
screenpy-selenium
```

1.5 Configuração Inicial

Nenhuma alteração agressiva no projeto é necessária.

ScreenPy roda direto com Pytest.

1.6 Implementação do Screenplay

1.6.1 Ability (Driver)

Arquivo:

tests/screenplay/abilities/browse_the_web.py

```
from screenpy import Ability
from screenpy_selenium.abilities import BrowseTheWeb as
ScreenPyBrowse


class BrowseTheWeb(Ability):
    @staticmethod
    def using(driver):
        return ScreenPyBrowse.using(driver)
```

1.6.2 Actor Factory

Arquivo:

tests/screenplay/actors/actor_factory.py

```
from screenpy import Actor
from tests.screenplay.abilities.browse_the_web import
BrowseTheWeb
from tests.fixtures.driver import create_driver


def create_actor(name="Usuário"):
    driver = create_driver()

    actor = Actor.named(name).who_can(
        BrowseTheWeb.using(driver)
     )

     actor.driver = driver  # fallback útil

     return actor
```

1.7 Reutilização das Pages (PONTO MAIS IMPORTANTE)

Você NÃO altera:

```
tests/pages/*
```

Mas agora as Pages serão usadas dentro de Tasks.

1.8 Tasks (Substituindo Transactions)

EXEMPLO: Login

Arquivo:

```
tests/screenplay/tasks/login.py
```

```python
from screenpy import Task
from tests.pages.login_page import LoginPage


class PerformLogin(Task):

    def __init__(self, user, password):
        self.user = user
        self.password = password

    def perform_as(self, actor):
        driver = actor.ability_to("BrowseTheWeb").browser

        page = LoginPage(driver)
        page.open()
        page.login(self.user, self.password)

    @staticmethod
    def with_credentials(user, password):
        return PerformLogin(user, password)
```

EXEMPLO: Add to Cart

```python
from screenpy import Task
from tests.pages.inventory_page import InventoryPage


class AddItemToCart(Task):
```

```
6
7       def perform_as(self, actor):
8           driver = actor.ability_to("BrowseTheWeb").browser
9           page = InventoryPage(driver)
10
11           page.add_first_item_to_cart()
```

1.9 Questions (Validações)

Arquivo:

tests/screenplay/questions/inventory.py

```
1  from screenpy import Question
2  from tests.pages.inventory_page import InventoryPage
3
4
5  class InventoryPageIsLoaded(Question):
6
7      def answered_by(self, actor):
8          driver = actor.ability_to("BrowseTheWeb").browser
9          page = InventoryPage(driver)
10
11          return page.is_loaded()
```

1.10 Testes Screenplay

EXEMPLO: Login

Arquivo:

tests/screenplay/tests/test_login_screenplay.py

```
1  from tests.screenplay.actors.actor_factory import create_actor
2  from tests.screenplay.tasks.login import PerformLogin
3  from tests.screenplay.questions.inventory import
InventoryPageIsLoaded
4
5
6  def test_login_com_sucesso():
7      actor = create_actor()
8
9      try:
```

```
        actor.attempts_to(
            PerformLogin.with_credentials("standard_user",
"secret_sauce")
        )

        assert InventoryPageIsLoaded().answered_by(actor)

    finally:
        actor.driver.quit()
```

1.11 Integração com Pytest Fixtures

Opção 1 (Recomendada para PoC)

Gerenciar driver manualmente no Actor

Opção 2 (Mais avançada)

Integrar com fixture:

```
@pytest.fixture
def actor():
    driver = create_driver()
    actor = Actor.named("User").who_can(
        BrowseTheWeb.using(driver)
    )
    yield actor
    driver.quit()
```

1.12 GitHub Actions

Adicionar step:

```
- name: Run Screenplay Tests
  run: pytest tests/screenplay/tests
```

Pipeline completo:

```
- name: Install dependencies
  run: pip install -r requirements.txt

- name: Run Specs
  run: pytest tests/specs
```

```
6
7  - name: Run Screenplay PoC
8    run: pytest tests/screenplay/tests
```

1.13 Estratégia de Convivência

Durante PoC:

Pasta	Função
specs	testes atuais
screenplay	PoC nova arquitetura

Sem impacto cruzado.

1.14 Documento da PoC

Arquivo:

```
1  tests/docs/screenplay_poc.md
```

Conteúdo:

```
1  # Screenplay PoC
2
3  ## Objetivo
4  Substituir Transactions por Tasks usando ScreenPy.
5
6  ## Benefícios esperados
7  - Redução de duplicação
8  - Aumento de legibilidade
9  - Melhor composição de cenários
10
11  ## Estratégia
12  - Não modificar Pages
13  - Isolar nova arquitetura
14
15  ## Execução
16  pytest tests/screenplay/tests
```

2. SUMÁRIO

O plano foi construído com base em:

- Reaproveitamento total de Page Objects
- Substituição progressiva de Transactions → Tasks
- Introdução de Actor + Ability para encapsular estado (driver)
- Integração simples com Pytest (sem ruptura)

Essa abordagem é **mais escalável que Behave isolado**, pois:

- reduz acoplamento
- promove reuso real de comportamento
- permite evoluir depois para BDD sem retrabalho

---

3. BOAS PRÁTICAS

3.1 NÃO transformar Tasks em "Transactions disfarçadas"

Evitar:

```
Task gigante que faz login + compra + validação
```

3.2 Criar Tasks pequenas e compostas

Exemplo ideal:

```
PerformLogin
AddItemToCart
ProceedToCheckout
FinishOrder
```

3.3 Questions devem ser puras

- sem efeitos colaterais
- apenas leitura

3.4 Actor SEMPRE com uma única responsabilidade

- representar persona real (User, Admin…)

3.5 Evolução futura

Depois da PoC você pode:

- integrar Screenplay + Behave
- remover Transactions completamente
- criar DSL de negócio

---

4. COMPARAÇÃO DIRETA (IMPORTANTE PARA DECISÃO)

Critério	Behave	Screenplay
Legibilidade negócio	Alta	Média
Reutilização código	Média	Alta
Manutenção	Média	Alta
Escalabilidade	Média	Alta
Curva aprendizado	Baixa	Média

---

5. PERGUNTA FINAL (IMPORTANTE)

Você quer evoluir a PoC para:

A) Screenplay puro (foco em arquitetura técnica)
B) Screenplay + Behave (BDD + arquitetura moderna)

A opção B exige um desenho híbrido mais complexo — posso montar esse plano na próxima etapa.